Functional Programming Practical Week 8 Part 2

Discriminated Unions Records and Pattern Matching

30 additional practical questions and tasks based on the detailed course plan. Use records and inheritance to simulate union types in C#, apply switch expressions and model domain alternatives. The final questions also cover the detailed lecture objective of using a C library from C#.

Use C# 12 and .NET 8. A record hierarchy models declared cases but is not an automatically closed or exhaustively checked union in this language version. Include a fallback for unexpected cases. Native questions provide small C interfaces to implement yourself and do not require an existing course library.

Optional and ungraded practice. During defence, you will need to implement one of these tasks or a similar task. Any practical task may appear in the endterm.

Basic tasks – Questions 01–05

Question 01 Two result cases

Define an abstract Creation record with Created(Product) and Rejected(string Reason) cases. Define Product with Id, Quantity and UnitPrice. Explain which payload belongs to each case.

Checks and discussion: Create one example of each case. A rejection must not require a dummy Product.

Question 02 A switch expression

Implement Describe(Creation) using a switch expression. Return the product id for Created and the reason for Rejected. Add a fallback that throws for an unexpected case.

Checks and discussion: Created with id "P1" returns "created:P1". Rejected("price") returns "rejected:price".

Question 03 Repairing a direct cast

A function casts every Creation to Created and reads Value.Id. Replace the cast with pattern matching and a meaningful rejection branch.

Checks and discussion: A Rejected input must produce the rejection description instead of InvalidCastException.

Question 04 An optional lookup

Model Lookup<T> with Found(T) and Missing cases. Implement a dictionary lookup returning the appropriate case.

Checks and discussion: A stored value of 0 becomes Found(0). An absent key becomes Missing.

Question 05 A positional pattern

Use a positional pattern to extract both quantity and price from a Product record and return their product. Explain where the positional components come from.

Checks and discussion: Product("P1", 2, 3.5m) gives 7m. Do not assume that the record itself checks validity.

Functional Programming Practical Week 8 Part 2

Basic tasks – Questions 06–10

Question 06 A property pattern

Write a function that recognises Created containing a product with Quantity > 0 and UnitPrice >= 0 using property and relational patterns.

Checks and discussion: Positive quantity with zero price is accepted. Zero quantity is rejected. A Rejected case is not accepted.

Question 07 Ordering switch arms

Write a switch expression with a specific zero-price product branch and a general Created branch. Explain why the specific branch must precede the broader matching branch.

Checks and discussion: A zero-price Created returns "free". A positive-price Created returns "priced".

Question 08 A null input policy

Implement Describe(Creation?) with a separate null branch. Distinguish a missing result reference from an explicit Rejected value.

Checks and discussion: null returns "no result". Rejected("id") returns an error description. Created is handled normally.

Question 09 Records compare values

Create two Product records with identical values and compare them with == and ReferenceEquals. Explain the results and contrast them with a plain class without custom equality.

Checks and discussion: Equal Product records compare equal even when they are separate instances.

Question 10 Copying a record

Use with to create a new Product whose quantity is increased by one. State whether the original changes and whether with automatically validates the new quantity.

Checks and discussion: Quantity 2 becomes 3 in the copy while the original stays 2. Demonstrate that a public init property can accept an invalid value.

Functional Programming Practical Week 8 Part 2

Applying the techniques – Questions 11–15

Question 11 An order state model

Define OrderState with Pending, Confirmed(decimal Total) and Cancelled(string Reason). Compare it with a model containing IsConfirmed, IsCancelled, nullable Total and nullable Reason.

Checks and discussion: List two contradictory flag combinations that the case-based representation avoids.

Question 12 Guarded confirmation

Implement Confirm(state, total) accepting only Pending and a nonnegative total. Return a new Confirmed case. Reject every other state through a stated exception or result contract.

Checks and discussion: Zero total is accepted. Negative total and confirming twice are rejected. The original Pending value remains unchanged.

Question 13 Guarded cancellation

Implement Cancel(state, reason) accepting only Pending and a nonblank trimmed reason. State how invalid transitions are reported.

Checks and discussion: " changed mind " stores "changed mind". Blank reason is rejected. Confirmed cannot be cancelled under this policy.

Question 14 Idempotent cancellation

Modify cancellation so an already Cancelled state returns unchanged and keeps its original reason. Compare this with rejecting every repeated call.

Checks and discussion: Cancelling twice with different reasons preserves the first cancellation reason.

Question 15 Rendering state-specific payloads

Implement a display function for every OrderState case. Confirmed should show Total, Cancelled should show Reason, and Pending should show neither.

Checks and discussion: Each state has a distinct output. Include an unexpected-case fallback.

Functional Programming Practical Week 8 Part 2

Applying the techniques – Questions 16–20

Question 16 One field should not mean two things

A Payment record uses Amount = -1 to mean cancelled and Amount = 0 to mean pending. Replace these sentinel values with explicit Payment cases.

Checks and discussion: Represent Pending, Cancelled(reason) and Paid(amount). Explain how an actual zero payment would be represented.

Question 17 Different payload types

Model a notification as Email(address, subject), Sms(number, text) or InApp(userId, text). Implement a preview function using pattern matching.

Checks and discussion: An Email preview must not read a phone number. Define the types of all fields explicitly.

Question 18 Nested alternatives

Model a download as NotStarted, InProgress(percent), Finished(FileInfoValue) or Failed(error). Define FileInfoValue as your own record with Name and SizeBytes.

Checks and discussion: Reject progress outside 0–100 through a factory. Finished must carry file information and Failed must carry a reason.

Question 19 An open hierarchy

Add a new case to an existing record hierarchy and examine a switch with a fallback. Explain why C# 12 does not automatically guarantee that all declared cases are handled.

Checks and discussion: Show the behaviour of the old switch when it receives the new case. Update the switch deliberately.

Question 20 A shallow-copy defect

A record contains a mutable string[] and is copied with an empty with expression. Demonstrate that changing an array element through the copy affects the original.

Checks and discussion: Repair the operation by cloning the array or choosing an immutable representation. State what the repair guarantees.

Functional Programming Practical Week 8 Part 2

Integration and review – Questions 21–25

Question 21 Typed commands

Define AddItem(Product), RemoveItem(string Id) and ClearCart command cases. Implement Apply(cart, command) returning a new cart rather than changing the input.

Checks and discussion: Removing an absent id leaves the contents unchanged. ClearCart returns an empty cart. State your duplicate-id policy.

Question 22 Typed state prerequisites

Compare Confirm(OrderState, total) with Confirm(Pending, total). Implement the latter and explain which invalid calls its parameter type prevents.

Checks and discussion: A Cancelled variable cannot be passed directly to the typed overload. Payload validation is still required.

Question 23 A union versus nullable payloads

Refactor a response with nullable Data and nullable Error into Success(Data) and Failure(Error). Explain what happens to the states where both fields are filled or neither is filled.

Checks and discussion: Construct one success and one failure without supplying irrelevant payloads.

Question 24 Separating business logic and display

Implement a validation function returning Created or Rejected and a separate renderer for those cases. Keep Console.WriteLine out of validation.

Checks and discussion: Use the same result to produce a console message and a short UI message without re-running validation.

Question 25 A domain rule in a factory

Build a factory for DateRange(Start, End) requiring End >= Start. Explain why a public record constructor and with can bypass this rule, then propose a controlled representation.

Checks and discussion: Equal dates are valid. End before Start is rejected. Describe how an update preserves the invariant.

Functional Programming Practical Week 8 Part 2

Integration and review – Questions 26–30

Question 26 A complete domain operation

Model a ticket as Open, Assigned(assignee) or Closed(resolution). Implement one allowed transition and explicitly reject the others according to your own written transition table.

Checks and discussion: Include a valid transition, a blank payload and a forbidden transition. Preserve the original ticket state.

Question 27 A native return code as a union

Given a C function returning 0 for success, 1 for bad input and 2 for overflow, design C# Success, InvalidInput and Overflow cases. Write the return-code mapping.

Checks and discussion: Map all three codes. Handle any other code as an unexpected native failure, with the code preserved.

Question 28 A small C binding

Write a C function int add_nonnegative(int a, int b, int* out_value) returning 0 on success, 1 for negative input and 2 for INT_MAX overflow. Write a C# P/Invoke declaration using int and out int.

Checks and discussion: Use an explicitly matching Cdecl calling convention and your chosen library name. Build your own C library before testing the binding.

Question 29 A managed wrapper for the C function

Wrap the binding from Question 28 in a C# method returning a record case. Read out_value only for return code 0. Distinguish loading or entry-point failures from expected native return codes.

Checks and discussion: Inputs 2 and 3 return a success carrying 5. Negative input and INT_MAX plus 1 return the documented failure cases.

Question 30 Domain logic with a native boundary

Design a small calculation feature that calls the C wrapper at its boundary and handles the resulting cases in pure C# logic. Keep native invocation separate from rendering.

Checks and discussion: Document the C signature, calling convention, cases and checks. Explain what remains testable without loading the C library.